

```

import React, { useState, useEffect, useRef } from 'react';
import { initializeApp } from 'firebase/app';
import { getAuth, signInWithCustomToken, signInAnonymously, onAuthStateChanged } from
'firebase/auth';
import { getFirestore, collection, doc, setDoc, onSnapshot, serverTimestamp, deleteDoc } from
'firebase/firestore';

// --- Icons ---
const SendIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><line x1="22" y1="2" x2="11" y2="13"></line><polyline points="22 2 15
22 11 13 2 9 22 2"></polyline></svg>;
const CheckCircleIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M22 11.08V12a10 10 0 0 1 1-5.93-9.14"></path><polyline
points="22 4 12 14.01 9 11.01"></polyline></svg>;
const CircleIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><circle cx="12" cy="12" r="10"></circle></svg>;
const TargetIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><circle cx="12" cy="12" r="10"></circle><circle cx="12" cy="12"
r="6"></circle><circle cx="12" cy="12" r="2"></circle></svg>;
const ListIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><line x1="8" y1="6" x2="21" y2="6"></line><line x1="8" y1="12" x2="21"
y2="12"></line><line x1="8" y1="18" x2="21" y2="18"></line><line x1="3" y1="6" x2="3.01"
y2="6"></line><line x1="3" y1="12" x2="3.01" y2="12"></line><line x1="3" y1="18" x2="3.01"
y2="18"></line></svg>;
const LoaderIcon = () => <svg className="animate-spin" xmlns="http://www.w3.org/2000/svg"
width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" strokeLinejoin="round"><line x1="12" y1="2" x2="12"
y2="6"></line><line x1="12" y1="18" x2="12" y2="22"></line><line x1="4.93" y1="4.93"
x2="7.76" y2="7.76"></line><line x1="16.24" y1="16.24" x2="19.07" y2="19.07"></line><line
x1="2" y1="12" x2="6" y2="12"></line><line x1="18" y1="12" x2="22" y2="12"></line><line
x1="4.93" y1="19.07" x2="7.76" y2="16.24"></line><line x1="16.24" y1="7.76" x2="19.07"
y2="4.93"></line></svg>;
const CopyIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><rect x="9" y="9" width="13" height="13" rx="2" ry="2"></rect><path
d="M5 15H4a2 2 0 0 1-2-2V4a2 2 0 0 1 2-2h9a2 2 0 0 1 2 2v1"></path></svg>;
const MessageIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2
2"></path></svg>;
const BoardIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><rect x="3" y="3" width="18" height="18" rx="2" ry="2"></rect><line

```

```

x1="9" y1="3" x2="9" y2="21"></line></svg>;
const SparkleIcon = ({ size = 18, className = "" }) => <svg
xmlns="http://www.w3.org/2000/svg" width={size} height={size} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
className={className}><path d="M12 3v1m0 16v1m9-9h-1M4 12H3m15.364
6.364l-.707-.707M6.343 6.343l-.707-.707m12.728 0l-.707-.707M6.343 17.657l-.707-.707M16
12a4 4 0 11-8 0 4 4 0 018 0z"></path></svg>;
const CpuIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><rect x="4" y="4" width="16" height="16" rx="2" ry="2"></rect><rect
x="9" y="9" width="6" height="6"></rect><line x1="9" y1="1" x2="9" y2="4"></line><line x1="15"
y1="1" x2="15" y2="4"></line><line x1="9" y1="20" x2="9" y2="23"></line><line x1="15"
y1="20" x2="15" y2="23"></line><line x1="20" y1="9" x2="23" y2="9"></line><line x1="20"
y1="14" x2="23" y2="14"></line><line x1="1" y1="9" x2="4" y2="9"></line><line x1="1" y1="14"
x2="4" y2="14"></line></svg>;
const LayersIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><polyline points="12 2 2 7 12 12 22 7 12 2"></polyline><polyline
points="2 17 12 22 22 17"></polyline><polyline points="2 12 17 22 12"></polyline></svg>;
const ObsidianIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0
3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4A2 2 0 0 0 21 16z"></path><polyline points="3.27
6.96 12 12.01 20.73 6.96"></polyline><line x1="12" y1="22.08" x2="12" y2="12"></line></svg>;
const MenuIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><line x1="3" y1="12" x2="21" y2="12"></line><line x1="3" y1="6"
x2="21" y2="6"></line><line x1="3" y1="18" x2="21" y2="18"></line></svg>;
const PlusIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12"
x2="19" y2="12"></line></svg>;
const TrashIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6v14a2 2 0 0
1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></path></svg>;
const CloudIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0
0-10z"></path></svg>;
const WarningIcon = () => <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0
1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"></path><line x1="12" y1="9" x2="12"
y2="13"></line><line x1="12" y1="17" x2="12.01" y2="17"></line></svg>;

```

```

// --- Firebase Configuration ---
// IMPORTANT FOR LOVABLE/CURSOR EXPORT:

```

```

// Replace the config object below with your actual Firebase project settings to enable
// permanent cloud saves.
const userProvidedFirebaseConfig = {
  apiKey: "YOUR_API_KEY_HERE",
  authDomain: "YOUR_AUTH_DOMAIN_HERE",
  projectId: "YOUR_PROJECT_ID_HERE",
  storageBucket: "YOUR_STORAGE_BUCKET_HERE",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID_HERE",
  appId: "YOUR_APP_ID_HERE"
};

let app, auth, db, appId;
let isDbConnected = false;

try {
  // Use the injected preview environment config first, OR fallback to the user's provided config
  const envConfig = typeof __firebase_config !== 'undefined' ? JSON.parse(__firebase_config) :
  null;
  const finalConfig = envConfig || (userProvidedFirebaseConfig.apiKey !==
  "YOUR_API_KEY_HERE" ? userProvidedFirebaseConfig : null);

  if (finalConfig) {
    app = initializeApp(finalConfig);
    auth = getAuth(app);
    db = getFirestore(app);
    appId = typeof __app_id !== 'undefined' ? __app_id : 'focusflow-local';
    isDbConnected = true;
  }
} catch (error) {
  console.error("Firebase init error:", error);
}

export default function App() {
  const [user, setUser] = useState(null);
  const [projects, setProjects] = useState([]);
  const [currentProjectId, setCurrentProjectId] = useState(null);
  const [isDrawerOpen, setIsDrawerOpen] = useState(false);
  const [isSaving, setIsSaving] = useState(false);

  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [activeTab, setActiveTab] = useState('chat');
  const [copyStatus, setCopyStatus] = useState(null);

  const [projectState, setProjectState] = useState({
    goal: "", outline: [], sessions: [], obsidian_markdown: ""
  });
};

```

```

const [completedSessions, setCompletedSessions] = useState(new Set());

const chatEndRef = useRef(null);
const textareaRef = useRef(null);
const saveTimeoutRef = useRef(null);

useEffect(() => {
  if (!auth || !isDbConnected) return;
  const initAuth = async () => {
    try {
      if (typeof __initial_auth_token !== 'undefined' && __initial_auth_token) {
        await signInWithCustomToken(auth, __initial_auth_token);
      } else {
        await signInAnonymously(auth);
      }
    } catch (e) {
      console.error("Auth error", e);
    }
  };
  initAuth();

  const unsubscribe = onAuthStateChanged(auth, setUser);
  return () => unsubscribe();
}, []);

useEffect(() => {
  if (!user || !db || !isDbConnected) return;

  const projectsRef = collection(db, 'artifacts', appId, 'users', user.uid, 'focusflow_projects');
  const unsubscribe = onSnapshot(projectsRef, (snapshot) => {
    const fetchedProjects = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
    fetchedProjects.sort((a, b) => {
      const timeA = a.updatedAt?.toMillis() || 0;
      const timeB = b.updatedAt?.toMillis() || 0;
      return timeB - timeA;
    });
    setProjects(fetchedProjects);
  }, (error) => {
    console.error("Error fetching projects:", error);
  });

  return () => unsubscribe();
}, [user]);

useEffect(() => {
  if (!currentProjectId || !user || !db || !isDbConnected) return;

  if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);

```

```

saveTimeoutRef.current = setTimeout(async () => {
  setIsSaving(true);
  try {
    const docRef = doc(db, 'artifacts', appId, 'users', user.uid, 'focusflow_projects',
currentProjectId);
    await setDoc(docRef, {
      title: projectState.goal ? projectState.goal.substring(0, 45) + "...": "New Project",
      projectState,
      messages,
      completedSessions: Array.from(completedSessions),
      updatedAt: serverTimestamp()
    }, { merge: true });
  } catch (err) {
    console.error("Error saving:", err);
  } finally {
    setIsSaving(false);
  }
}, 1500);

return () => clearTimeout(saveTimeoutRef.current);
}, [messages, projectState, completedSessions, currentProjectId, user]);

```

```

const createNewProject = () => {
  const newId = crypto.randomUUID();
  setCurrentProjectId(newId);
  setMessages([]);
  setProjectState({ goal: "", outline: [], sessions: [], obsidian_markdown: "" });
  setCompletedSessions(new Set());
  setIsDrawerOpen(false);
  setActiveTab('chat');
};

```

```

const loadProject = (project) => {
  setCurrentProjectId(project.id);
  setMessages(project.messages || []);
  setProjectState(project.projectState || { goal: "", outline: [], sessions: [], obsidian_markdown: ""
});
  setCompletedSessions(new Set(project.completedSessions || []));
  setIsDrawerOpen(false);
};

```

```

const deleteProject = async (e, id) => {
  e.stopPropagation();
  if (!user || !db || !isDbConnected) return;
  try {
    await deleteDoc(doc(db, 'artifacts', appId, 'users', user.uid, 'focusflow_projects', id));

```

```

    if (currentProjectId === id) {
      createNewProject();
    }
  } catch (err) {
    console.error("Error deleting:", err);
  }
};

useEffect(() => {
  if (!currentProjectId) createNewProject();
}, []);

useEffect(() => {
  if (activeTab === 'chat' && messages.length > 0) {
    chatEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }
}, [messages, activeTab]);

const handleInput = (e) => {
  setInput(e.target.value);
  if (textareaRef.current) {
    textareaRef.current.style.height = 'auto';
    textareaRef.current.style.height = `${Math.min(textareaRef.current.scrollHeight, 150)}px`;
  }
};

const toggleSession = (id) => {
  const newSet = new Set(completedSessions);
  if (newSet.has(id)) newSet.delete(id);
  else newSet.add(id);
  setCompletedSessions(newSet);
};

const copyToClipboard = (text, type) => {
  const textArea = document.createElement("textarea");
  textArea.value = text;
  textArea.style.position = "fixed";
  textArea.style.left = "-999999px";
  document.body.appendChild(textArea);
  textArea.focus();
  textArea.select();
  try {
    document.execCommand('copy');
    setCopyStatus(type);
    setTimeout(() => setCopyStatus(null), 2000);
  } catch (err) {
    console.error('Unable to copy', err);
  }
}

```

```

document.body.removeChild(textArea);
};

const openInObsidian = () => {
  if (!projectState.obsidian_markdown) return;
  const rawTitle = projectState.goal ? projectState.goal.substring(0, 30) : "Plan";
  const title = "FocusFlow - " + rawTitle.replace(/[^a-zA-Z0-9 ]/g, "");
  const content = projectState.obsidian_markdown;
  const uri =
`obsidian://new?name=${encodeURIComponent(title)}&content=${encodeURIComponent(cont
ent)}`;
  window.location.href = uri;
  copyToClipboard(content, 'obsidian');
};

const handleSend = async () => {
  if (!input.trim() || isLoading) return;

  const userText = input.trim();
  setInput("");

  if (textareaRef.current) {
    textareaRef.current.style.height = 'auto';
  }

  const newMessages = [...messages, { role: "user", content: userText }];
  setMessages(newMessages);
  setIsLoading(true);

  try {
    const apiKey = "";
    const systemInstruction = `
      You are an expert Socratic project manager, ADHD executive function coach, and
      Tri-Model AI Orchestrator.
      Your goal is to help the user break a raw idea into 45-90 min 'sessions' and route each to
      the optimal AI model.
    `;

```

CRITICAL TRI-MODEL ROUTING RULES:

For every session, you MUST recommend the best AI tool and Tech Stack rules:

1. GEMINI (3.1 Series / Previews): Route here for unstructured multimodal research, analyzing videos/mockups, massive context ingestion (1M-2M+ tokens), and live web research.
2. OPENAI (GPT-5.5 / Previews): Route here for core engineering, complex backend logic, multi-file refactoring, abstract math, and strict API orchestrations.
3. Z.AI (GLM Series): Route here for Phase 3 "Vibe Coding" (high-aesthetic modern UI/frontend generation via GLM-4.7), long-running terminal agent loops utilizing "Preserved Thinking" (preventing context amnesia), and rigid image OCR schema extraction.

TECH STACK RULES:

- For Zero-to-One New Apps: Default to the optimal stack (Next.js/React/Tailwind/shadcn) UNLESS there's a reason not to. Note that GLM-4.7 excels at "Vibe Coding" for this layer.
- For Existing Codebases: Default to "audit and adapt". Only surface a refactor if broken.

5-PHASE APPROACH:

1. Define Goal.
2. Rough Outline.
3. Break into Sessions (Include exact AI Model Tier & Stack details in the JSON).
4. Define Win Conditions & Deliverables.
5. Generate the Obsidian Master Plan (The overall lifecycle mapped out with checkboxes '[-]'). Fill 'obsidian_markdown'.

- Output in JSON format. Ask 1-2 questions at a time. Socratic tone.

`;

```
const conversationHistory = newMessages.map(m => ({
  role: m.role === 'assistant' ? 'model' : 'user',
  parts: [{ text: m.content }]
}));
```

```
const payload = {
  systemInstruction: { parts: [{ text: systemInstruction }] },
  contents: conversationHistory,
  generationConfig: {
    responseMimeType: "application/json",
    responseSchema: {
      type: "OBJECT",
      properties: {
        chat_response: { type: "STRING" },
        project_state: {
          type: "OBJECT",
          properties: {
            goal: { type: "STRING" },
            outline: { type: "ARRAY", items: { type: "STRING" } },
            obsidian_markdown: { type: "STRING" },
            sessions: {
              type: "ARRAY",
              items: {
                type: "OBJECT",
                properties: {
                  id: { type: "STRING" },
                  title: { type: "STRING" },
                  description: { type: "STRING" },
                  recommended_agent: { type: "STRING", description: "e.g., 'GLM-4.7 (Vibe Coding)', 'OpenAI Preview', 'Gemini 3.1 Pro'" },
                  tech_stack_rules: { type: "STRING" },
                  win_condition: { type: "STRING" },
```



```

        deliverables: { type: "ARRAY", items: { type: "STRING" } },
        dependencies: { type: "ARRAY", items: { type: "STRING" } }
    }
}
}
}
},
required: ["chat_response", "project_state"]
}
}
};

let result;
let retries = 0;
const maxRetries = 5;
const delays = [1000, 2000, 4000, 8000, 16000];

while (retries <= maxRetries) {
    try {
        const response = await
fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-preview-09-2
025:generateContent?key=${apiKey}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
    });

    if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);

    const data = await response.json();
    const textResponse = data.candidates?.[0]?.content?.parts?.[0]?.text;

    if (!textResponse) throw new Error("Empty response from API");

    result = JSON.parse(textResponse);
    break;

    } catch (err) {
        if (retries === maxRetries) {
            setMessages([...newMessages, {
                role: "assistant",
                content: "I'm having trouble processing that right now. Could you try again in a
moment?"
            }]);
            setIsLoading(false);
            return;
        }
    }
}

```

```

        await new Promise(r => setTimeout(r, delays[retries]));
        retries++;
    }
}

if (result) {
    setMessages([...newMessages, { role: "assistant", content: result.chat_response }]);
    if (result.project_state) {
        setProjectState(prevState => ({
            ...prevState,
            ...result.project_state
        }));
    }
}

} catch (error) {
    console.error("Chat Error:", error);
} finally {
    setIsLoading(false);
}
};

const handleKeyDown = (e) => {
    if (e.key === 'Enter' && !e.shiftKey) {
        e.preventDefault();
        handleSend();
    }
};

return (
    <div className="absolute inset-0 flex flex-col bg-[#0E1117] text-gray-200 font-sans overflow-hidden">

        {/* Sliding Projects Drawer Overlay */}
        {isDrawerOpen && (
            <div
                className="fixed inset-0 bg-black/60 backdrop-blur-sm z-40 transition-opacity"
                onClick={() => setIsDrawerOpen(false)}
            />
        )}

        {/* Projects Drawer Sidebar */}
        <div className={`fixed top-0 left-0 bottom-0 w-80 bg-[#161B22] border-r border-[#30363D]
z-50 transform transition-transform duration-300 ease-in-out flex flex-col shadow-2xl
    ${isDrawerOpen ? 'translate-x-0' : '-translate-x-full'}`}
        >
            <div className="p-4 border-b border-[#30363D] flex items-center justify-between">
                <div className="flex items-center gap-2 text-gray-200 font-bold">

```

```

    <LayersIcon /> Saved Plans
  </div>
  <button onClick={() => setIsDrawerOpen(false)} className="p-1 hover:bg-[#30363D]
rounded-md text-gray-400">
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><line x1="18" y1="6" x2="6" y2="18"></line><line x1="6" y1="6" x2="18"
y2="18"></line></svg>
  </button>
</div>

<div className="p-4 border-b border-[#30363D]">
  <button
    onClick={createNewProject}
    className="w-full flex items-center justify-center gap-2 py-2 bg-[#238636]
hover:bg-[#2EA043] text-white rounded-md font-bold text-sm transition-colors"
  >
    <PlusIcon /> New Project
  </button>
</div>

<div className="flex-1 overflow-y-auto p-2 space-y-1 custom-scrollbar">
  {!isDbConnected ? (
    <div className="text-center p-6 text-gray-400 text-sm flex flex-col items-center
gap-3">
      <div className="p-3 bg-red-900/20 rounded-full text-red-400"><WarningIcon /></div>
      <p>Database not connected.</p>
      <p className="text-xs text-gray-500">Provide your Firebase config in the code to
enable persistent saving in Lovable/Cursor.</p>
    </div>
  ) : projects.length === 0 ? (
    <div className="text-center p-4 text-gray-500 text-sm italic">
      No saved projects yet. Start chatting to auto-save!
    </div>
  ) : (
    projects.map(p => (
      <div
        key={p.id}
        onClick={() => loadProject(p)}
        className={`w-full text-left p-3 rounded-md transition-colors cursor-pointer group flex
items-start justify-between gap-2
          ${currentProjectId === p.id ? 'bg-[#1F6FEB]/20 border border-[#1F6FEB]/50' :
'hover:bg-[#30363D] border border-transparent'}
        >
          <div className="flex-1 overflow-hidden">
            <div className={`text-sm font-bold truncate ${currentProjectId === p.id ?
'text-[#58A6FF]' : 'text-gray-300'}>
              {p.title}

```

```

        </div>
        <div className="text-[10px] text-gray-500 mt-1">
          {p.updatedAt ? new Date(p.updatedAt.toDate()).toLocaleDateString() : 'Just now'}
        </div>
      </div>
      <button
        onClick={(e) => deleteProject(e, p.id)}
        className="p-1.5 text-gray-500 hover:text-red-400 hover:bg-red-900/20 rounded
opacity-0 group-hover:opacity-100 transition-opacity"
        title="Delete Plan"
      >
        <TrashIcon />
      </button>
    </div>
  ))
}
</div>

{ /* Sync Status Footer */}
<div className={`p-3 border-t border-[#30363D] flex items-center justify-center gap-2
text-[11px] font-mono
${!isDbConnected ? 'bg-[#0D1117] text-gray-500' : 'bg-red-950/30 text-red-400
border-t-red-900/50'}`}>
  {!isDbConnected ? (
    <><WarningIcon /> Local Memory Only (Clears on Refresh)</>
  ) : isSaving ? (
    <><LoaderIcon /> Syncing to Cloud...</>
  ) : (
    <><CloudIcon /> Cloud Synced</>
  )}
</div>
</div>

{ /* Global Header */}
<div className="flex-none w-full bg-[#161B22] border-b border-[#30363D] p-2 md:p-3 flex
items-center justify-between z-10">
  <div className="flex items-center gap-3">
    <button
      onClick={() => setIsDrawerOpen(true)}
      className="p-2 bg-[#21262D] hover:bg-[#30363D] border border-[#30363D]
rounded-md text-gray-300 transition-colors relative"
      title="Open Saved Plans"
    >
      <MenuIcon />
      {isDbConnected && <span className="absolute -top-1 -right-1 w-2.5 h-2.5 bg-red-500
rounded-full border-2 border-[#161B22]"></span>}
    </button>

```

```

<div className="flex items-center gap-2">
  <div className="bg-[#238636]/20 p-1.5 rounded-lg text-[#238636] hidden sm:block">
    <SparkleIcon size={16} />
  </div>
</div>
<div>
  <h1 className="text-base font-bold text-gray-100 leading-tight">FocusFlow</h1>
  <p className="text-[10px] text-gray-400 font-mono hidden sm:block">TRI_MODEL_ORCHESTRATOR</p>
</div>
</div>

{/* Mobile Tab Toggle */}
<div className="md:hidden flex gap-1">
  <button
    onClick={() => setActiveTab('chat')}
    className={`px-3 py-1.5 rounded text-xs font-bold transition-all ${activeTab === 'chat' ?
'bg-[#238636] text-white' : 'bg-[#21262D] text-gray-400 border border-[#30363D]'}`}
  >
    Chat
  </button>
  <button
    onClick={() => setActiveTab('board')}
    className={`px-3 py-1.5 rounded text-xs font-bold transition-all ${activeTab === 'board'
? 'bg-[#1F6FEB] text-white' : 'bg-[#21262D] text-gray-400 border border-[#30363D]'}`}
  >
    Board
    {projectState.sessions?.length > 0 && activeTab !== 'board' && (
      <span className="inline-block w-1.5 h-1.5 rounded-full bg-[#E3B341] ml-1 mb-0.5
animate-pulse"></span>
    )}
  </button>
</div>
</div>

{/* Main Content */}
<div className="flex-1 flex flex-col md:flex-row overflow-hidden w-full max-w-7xl mx-auto
relative">

  {/* Left Panel: Chat Interface */}
  <div className={` ${activeTab === 'chat' ? 'flex' : 'hidden'} md:flex w-full md:w-[45%]
flex-col border-r border-[#30363D] bg-[#0D1117] h-full`}>

    <div className="flex-1 overflow-y-auto p-4 md:p-6 space-y-8 custom-scrollbar">

      {messages.length === 0 && !isLoading && (
        <div className="h-full flex flex-col items-center justify-center text-center opacity-70
px-4 mt-12 md:mt-0">

```

```

<div className="bg-[#238636]/10 p-4 rounded-full mb-6">
  <SparkleIcon size={40} className="text-[#238636]" />
</div>
<h2 className="text-2xl font-bold mb-3 text-gray-100">Tri-Model Orchestrator</h2>
<p className="text-[#8B949E] text-[15px] max-w-sm leading-relaxed mb-4">
  I'm your AI Project Manager. I'll break down your idea and route tasks across <span
className="text-blue-400 font-bold">OpenAI Previews</span>, <span
className="text-orange-400 font-bold">Gemini 3.1</span>, and <span
className="text-teal-400 font-bold">Z.AI</span> based on their specific strengths.
</p>
  <!isDbConnected && (
    <p className="text-red-400 text-xs font-mono bg-red-900/20 p-2 rounded border
border-red-900/50">
      ⚠ Database disabled. Chat history will erase on refresh.
    </p>
  )}
</div>
)}

{messages.map((msg, idx) => (
  <div key={idx} className="flex flex-col space-y-2 max-w-3xl mx-auto w-full">
    <div className="flex items-center gap-2 text-[13px] font-bold uppercase
tracking-wider text-gray-500">
      {msg.role === 'user' ? (
        <><div className="w-5 h-5 rounded-full bg-[#1F6FEB] flex items-center
justify-center text-white text-[10px]">U</div> You</>
      ) : (
        <><div className="w-5 h-5 rounded-full bg-[#238636] flex items-center
justify-center text-white text-[10px]">AI</div> Orchestrator</>
      )}
    </div>
    <div className={`text-[15px] leading-relaxed whitespace-pre-wrap pl-7 ${msg.role
=== 'user' ? 'text-gray-300' : 'text-gray-100'}`}>
      {msg.content}
    </div>
  </div>
))}
{isLoading && (
  <div className="flex flex-col space-y-2 max-w-3xl mx-auto w-full">
    <div className="flex items-center gap-2 text-[13px] font-bold uppercase
tracking-wider text-gray-500">
      <div className="w-5 h-5 rounded-full bg-[#238636] flex items-center justify-center
text-white text-[10px]">AI</div> Orchestrator
    </div>
    <div className="text-[15px] pl-7 text-gray-400 flex items-center gap-2">
      <LoaderIcon /> Synthesizing & Routing...
    </div>
  </div>
)}

```

```

    })
    <div ref={chatEndRef} className="h-4" />
  </div>

  <div className="flex-none p-4 pb-[calc(1rem+env(safe-area-inset-bottom))] md:p-6
border-t border-[#30363D] md:border-t-0 bg-[#0D1117] md:bg-gradient-to-t from-[#0D1117]
via-[#0D1117] to-transparent">
  <div className="max-w-3xl mx-auto relative group">
    <div className="absolute -inset-0.5 bg-gradient-to-r from-[#238636] to-[#1F6FEB]
rounded-xl blur opacity-20 group-focus-within:opacity-40 transition duration-500"></div>
    <div className="relative flex bg-[#161B22] border border-[#30363D] rounded-xl
shadow-2xl focus-within:border-[#58A6FF] transition-colors">
      <textarea
        ref={textareaRef}
        value={input}
        onChange={handleInput}
        onKeyDown={handleKeyDown}
        placeholder={messages.length === 0 ? "Describe your project or codebase..." :
"Type your answer..."}
        className="w-full bg-transparent text-gray-100 text-[16px] placeholder-[#8B949E]
p-4 pr-14 outline-none resize-none min-h-[56px] rounded-xl"
        rows={1}
        disabled={isLoading}
        style={{ maxHeight: '150px' }}
      />
      <button
        onClick={handleSend}
        disabled={isLoading || !input.trim()}
        className="absolute right-2 bottom-2 p-2.5 bg-[#238636] hover:bg-[#2EA043]
disabled:bg-[#30363D] disabled:text-gray-500 text-white rounded-lg transition-colors flex
items-center justify-center active:scale-95"
      >
        <SendIcon />
      </button>
    </div>
    <div className="text-center mt-2 text-[11px] text-gray-500 font-mono hidden
md:block">
      Press Enter to send, Shift+Enter for new line
    </div>
  </div>
</div>
</div>
</div>

  { /* Right Panel: Dynamic Project Board */ }
  <div className={` ${activeTab === 'board' ? 'flex' : 'hidden'} md:flex w-full md:w-[55%]
flex-col bg-[#0E1117] h-full`}>

    <div className="flex-none p-4 md:p-5 border-b border-[#30363D] flex justify-between

```

```

items-center bg-[#0E1117] z-10 gap-2 overflow-x-auto custom-scrollbar">
  <h2 className="text-lg md:text-xl font-bold text-gray-100 flex-shrink-0">Blueprint</h2>

  <div className="flex gap-2 flex-shrink-0">
    <button
      onClick={() => copyToClipboard(JSON.stringify(projectState, null, 2), 'json')}
      className="flex items-center gap-2 text-xs font-bold text-gray-300 hover:text-white
bg-[#21262D] hover:bg-[#30363D] px-3 py-2 rounded-md transition-colors border
border-[#30363D]"
      title="Copy JSON State"
    >
      {copyStatus === 'json' ? <CheckCircleIcon size={14}/> : <CopyIcon />}
      <span className="hidden sm:inline">JSON</span>
    </button>

    <button
      onClick={openInObsidian}
      disabled={!projectState.obsidian_markdown}
      className={flex items-center gap-2 text-xs font-bold px-3 py-2 rounded-md
transition-colors border shadow-sm
      ${projectState.obsidian_markdown
        ? 'bg-[#7C3AED] text-white border-[#7C3AED] hover:bg-[#6D28D9]'
        : 'bg-[#21262D] text-gray-600 border-[#30363D] cursor-not-allowed'}}
      title="Open directly in Obsidian App"
    >
      {copyStatus === 'obsidian' ? <CheckCircleIcon size={14}/> : <ObsidianIcon />}
      <span>Open in Obsidian</span>
    </button>
  </div>
</div>

<div className="flex-1 overflow-y-auto p-4 md:p-8 space-y-8
pb-[calc(2rem+env(safe-area-inset-bottom))]" custom-scrollbar">

  <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
    <section className="bg-[#161B22] rounded-xl p-5 border border-[#30363D]">
      <h3 className="text-[12px] font-bold text-[#58A6FF] uppercase tracking-widest mb-3
flex items-center gap-2">
        <TargetIcon /> Core Goal
      </h3>
      {projectState.goal ? (
        <p className="text-gray-200 text-[15px] leading-relaxed
font-medium">{projectState.goal}</p>
      ) : (
        <p className="text-[#8B949E] italic text-[14px]">Waiting for definition...</p>
      )}
    </section>

```



```

<section className="bg-[#161B22] rounded-xl p-5 border border-[#30363D]">
  <h3 className="text-[12px] font-bold text-[#3FB950] uppercase tracking-widest mb-4
flex items-center gap-2">
    <ListIcon /> Structure
  </h3>
  {projectState.outline && projectState.outline.length > 0 ? (
    <ol className="list-decimal list-outside text-gray-300 space-y-2 text-[14px] ml-5">
      {projectState.outline.map((step, idx) => (
        <li key={idx} className="pl-1">{step}</li>
      ))}
    </ol>
  ) : (
    <p className="text-[#8B949E] italic text-[14px]">Awaiting structure outline...</p>
  )}
</section>
</div>

```

```

{projectState.obsidian_markdown && (
  <section className="bg-gradient-to-r from-[#7C3AED]/20 to-[#4C1D95]/10
rounded-xl p-5 border border-[#7C3AED]/40 flex items-center gap-4 shadow-lg">
    <div className="p-3 bg-[#7C3AED] rounded-full text-white shadow-md">
      <ObsidianIcon />
    </div>
    <div className="flex-1">
      <h3 className="text-[15px] font-bold text-gray-100 mb-1">Master Plan
Ready!</h3>
      <p className="text-gray-400 text-[13px] leading-relaxed">
        Click "Open in Obsidian" above to instantly create a new note in your local vault
with all tasks and checkboxes pre-formatted.
      </p>
    </div>
  </section>
)}

```

```

<section>
  <h3 className="text-[12px] font-bold text-[#E3B341] uppercase tracking-widest mb-5
flex items-center gap-2 px-1">
    <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24
24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round"><path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4a2 2 0 0 0
3 8v8a2 2 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4a2 2 0 0 0 21 16z"></path><polyline points="3.27
6.96 12 12.01 20.73 6.96"></polyline><line x1="12" y1="22.08" x2="12" y2="12"></line></svg>
    Execution Chunks & Routing
  </h3>

```

```

{projectState.sessions && projectState.sessions.length > 0 ? (
  <div className="space-y-4">
    {projectState.sessions.map((session) => {

```

```

const isCompleted = completedSessions.has(session.id);
return (
  <div
    key={session.id}
    className={`relative rounded-xl border transition-all duration-200
      ${isCompleted
        ? 'bg-[#161B22]/50 border-[#238636]/30 opacity-70'
        : 'bg-[#161B22] border-[#30363D] hover:border-[#58A6FF]/50 shadow-sm'}`}
  >
    <div className="p-5 md:p-6">
      <div className="flex items-start justify-between gap-4">
        <div className="flex-1">
          <div className="flex flex-wrap items-center gap-3 mb-2">
            <span className="px-2 py-1 rounded bg-[#0D1117] text-[#8B949E] border
border-[#30363D] text-[11px] font-mono font-bold">
              {session.id}
            </span>
            <h4 className={`font-bold text-lg ${isCompleted ? 'text-[#8B949E]
line-through' : 'text-gray-200'}`}>
              {session.title}
            </h4>
          </div>
          <p className={`text-[14px] mb-4 leading-relaxed ${isCompleted ?
'text-[#8B949E]' : 'text-gray-400'}`}>
            {session.description}
          </p>

          <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 mb-5">
            {session.recommended_agent && (
              <div className="flex items-start gap-2 bg-[#0D1117] p-2.5 rounded-lg
border border-[#30363D]">
                <div className="text-purple-400 mt-0.5"><Cpulcon /></div>
                <div>
                  <div className="text-[10px] uppercase text-gray-500 font-bold
mb-0.5">Route To</div>
                  <div className={`text-[13px] font-medium ${isCompleted ?
'text-[#8B949E]' : 'text-gray-200'}`}>{session.recommended_agent}</div>
                </div>
              </div>
            )}
            {session.tech_stack_rules && (
              <div className="flex items-start gap-2 bg-[#0D1117] p-2.5 rounded-lg
border border-[#30363D]">
                <div className="text-blue-400 mt-0.5"><LayersIcon /></div>
                <div>
                  <div className="text-[10px] uppercase text-gray-500 font-bold
mb-0.5">Stack Rules</div>
                  <div className={`text-[13px] font-medium ${isCompleted ?

```

```

'text-[#8B949E]' : 'text-gray-200'}}>{session.tech_stack_rules}</div>
    </div>
  </div>
  ))
</div>

    {session.dependencies && session.dependencies.length > 0 && (
    <div className="flex items-center gap-2 mb-4">
      <span className="text-[10px] uppercase text-[#8B949E]
font-bold">Requires:</span>
      <div className="flex gap-1.5">
        {session.dependencies.map(dep => (
          <span key={dep} className="px-1.5 py-0.5 rounded text-[10px]
font-mono font-bold bg-[#E3B341]/10 text-[#E3B341] border border-[#E3B341]/20">
            {dep}
          </span>
        ))}
      </div>
    </div>
  ))

    <div className={`rounded-lg p-4 border ${isCompleted ? 'bg-[#0D1117]/50
border-[#30363D]/50' : 'bg-[#0D1117] border-[#30363D]'}`}>
      <div className="text-[10px] font-bold text-[#8B949E] uppercase
tracking-widest mb-1.5">Win Condition</div>
      <p className={`text-[14px] font-medium ${isCompleted ? 'text-[#8B949E]' :
'text-gray-300'}`}>
        {session.win_condition}
      </p>

      {session.deliverables && session.deliverables.length > 0 && (
        <div className="mt-3 pt-3 border-t border-[#30363D]/50">
          <div className="text-[10px] font-bold text-[#8B949E] uppercase
tracking-widest mb-1.5">Deliverables</div>
          <ul className={`text-[13px] list-disc list-outside ml-4 space-y-1
${isCompleted ? 'text-[#8B949E]' : 'text-gray-400'}`}>
            {session.deliverables.map((del, i) => <li key={i}
className="pl-1">{del}</li>)}
          </ul>
        </div>
      )}
    </div>
  </div>

  <button
    onClick={() => toggleSession(session.id)}
    className={`flex-shrink-0 p-3 -mt-2 -mr-2 rounded-full transition-all
active:scale-95

```

```

        ${isCompleted ? 'text-[#238636] hover:text-[#2EA043] bg-[#238636]/10' :
'text-[#8B949E] hover:text-[#58A6FF] bg-[#21262D] hover:bg-[#30363D]`}
    >
        {isCompleted ? <CheckCircleIcon /> : <CircleIcon />}
    </button>
</div>
</div>
</div>
);
}}
</div>
): (
    <div className="text-center py-16 px-6 border border-dashed border-[#30363D]
rounded-xl bg-[#161B22]/30">
        <p className="text-[#8B949E] text-[14px]">
            Tasks and routing assignments will generate here dynamically.
        </p>
    </div>
    )}
</section>
</div>
</div>
</div>

<style dangerouslySetInnerHTML={{__html: `
.custom-scrollbar::-webkit-scrollbar { width: 6px; height: 6px; }
.custom-scrollbar::-webkit-scrollbar-track { background: transparent; }
.custom-scrollbar::-webkit-scrollbar-thumb { background: #30363D; border-radius: 4px; }
.custom-scrollbar::-webkit-scrollbar-thumb:hover { background: #8B949E; }
`}} />
</div>
);
}

```